APACHE KAFKA

KRaft Mode — Complete Junior → Middle Guide

Docker · KRaft · Java · Spring Boot

prepared for vbforge · Kyiv, Ukraine

PHASE 1

Kafka Fundamentals & Architecture

What Is Apache Kafka?

Apache Kafka is a distributed event streaming platform. Think of it as a high-performance, fault-tolerant message bus that decouples producers (who write data) from consumers (who read data). It was originally built at LinkedIn and is now the backbone of real-time data pipelines at thousands of companies.

 Mental model: Kafka is a commit log on steroids. Events are written in order, stored durably, and can be replayed at any time by any consumer.

KRaft Mode — Why It Matters

Traditionally Kafka required ZooKeeper for cluster coordination (leader election, metadata storage). From Kafka 3.3+, KRaft (Kafka Raft) mode replaces ZooKeeper entirely. The Kafka brokers themselves handle consensus using the Raft algorithm.

Feature	ZooKeeper mode vs KRaft mode
External dependency	ZooKeeper required (separate process) → None in KRaft
Metadata	Stored in ZooKeeper → Stored in internal __cluster_metadata topic
Scalability	ZK limits ~200k partitions → Millions of partitions possible
Deployment	Two systems to manage → Single system
Status	Deprecated (Kafka 4.x removes ZK) → Default & future

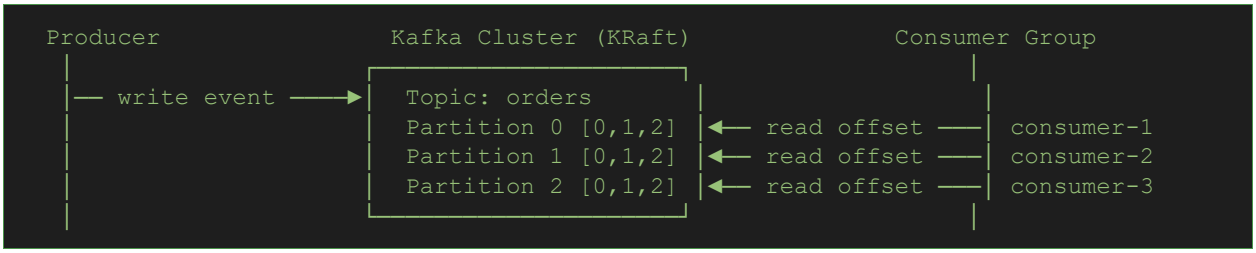
Core Concepts — Know These Cold

Term	What it means
Event / Record	The atomic unit of data. Has: key, value, timestamp, headers.
Topic	A named stream of events. Like a database table but append-only.
Partition	A topic is split into N partitions. Each is an ordered, immutable log on disk.
Offset	The position of a record within a partition. Monotonically increasing integer.
Producer	Application that writes events to a topic.
Consumer	Application that reads events from a topic.

Consumer Group	A group of consumers that collectively consume a topic. Each partition is read by exactly ONE consumer in the group.
Broker	A Kafka server. Stores partition data and serves clients.
Leader / Follower	For each partition, one broker is the leader (handles reads/writes). Others are followers (replicas).
Replication Factor	How many copies of each partition exist across brokers.
ISR	In-Sync Replicas — followers that are caught up to the leader. Writes are committed only when all ISR members acknowledge.
Retention	How long Kafka keeps events: by time (e.g. 7 days) or by size.

How Data Flows — The Big Picture

Producer writes to **Topic** → **Partition** (assigned by key hash or round-robin) → stored on **Broker (Leader)** → replicated to **Followers** → **Consumer Group** reads, commits offset to **__consumer_offsets** topic.



Key rule: A consumer group with 3 consumers on a topic with 2 partitions → one consumer will be idle. Partitions ≥ consumers for maximum parallelism.

KRaft Roles

In KRaft mode, each broker has a role defined in the config:

Role	Responsibility
controller	Participates in Raft quorum. Manages cluster metadata (topic creation, leader election). Does NOT serve client data.
broker	Stores partition data, serves producer/consumer requests.
broker,controller	Combined role — common in single-node dev setups (your Docker image).

✓PRACTICE TASK 1: Inspect Your Running Kafka Container

Goal: Get comfortable with your Docker Kafka setup and verify KRaft is running correctly.

Step 1 — Start your container:

```
docker run -d \
  --name kafka-kraft \
  -p 9092:9092 \
  confluentinc/cp-kafka:latest
```

Step 2 — Verify KRaft mode is active:

```
# Check the Kafka startup logs for KRaft confirmation
docker logs kafka-kraft | grep -i 'kraft\|raft\|controller'

# You should see lines like:
# [KafkaServer id=1] started (kafka.server.KafkaServer)
# Completed transition to running; controller epoch ...
```

Step 3 — Drop into the container shell:

```
docker exec -it kafka-kraft /bin/bash
```

Step 4 — List topics (should be empty at first):

```
kafka-topics --bootstrap-server localhost:9092 --list
```

Step 5 — Describe the cluster metadata topic:

```
kafka-topics --bootstrap-server localhost:9092 \
  --describe --topic __cluster_metadata

# This is the internal KRaft topic. No ZooKeeper equivalent exists.
```

Expected outcome: You can shell in, list topics, and confirm __cluster_metadata exists — proof KRaft is managing the cluster.

PHASE 2

Topics, Producers & Consumers via CLI

Working with Topics

Everything in Kafka lives in topics. Before writing Java code, master the CLI tools — they're available inside your container.

Create a Topic

```
docker exec -it kafka-kraft \
  kafka-topics --bootstrap-server localhost:9092 \
  --create \
  --topic user-events \
  --partitions 3 \
  --replication-factor 1

# replication-factor 1 = single broker (fine for dev)
# partitions 3 = max 3 parallel consumers in a group
```

Describe a Topic

```
docker exec -it kafka-kraft \
  kafka-topics --bootstrap-server localhost:9092 \
  --describe --topic user-events

# Output shows: Leader, Replicas, ISR per partition
```

Delete a Topic

```
docker exec -it kafka-kraft \
  kafka-topics --bootstrap-server localhost:9092 \
  --delete --topic user-events
```

Producing Messages via CLI

```
# Start a console producer (type messages, press Enter to send)
docker exec -it kafka-kraft \
  kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic user-events

> hello kafka
> {"userId": 1, "action": "login"}
^C  # Ctrl+C to exit
```

```
# Producer with keys (key:value separated by :)
docker exec -it kafka-kraft \
  kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic user-events \
  --property parse.key=true \
  --property key.separator=:

> user-1:{"userId": 1, "action": "login"}
> user-2:{"userId": 2, "action": "purchase"}
```

 Key routing: Records with the same key always go to the same partition. This guarantees ordering PER KEY. Example: all events for user-1 land in partition 0, always in order.

Consuming Messages via CLI

```
# Read from the beginning (--from-beginning replays all stored events)
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic user-events \
  --from-beginning

# Read with key display
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic user-events \
  --from-beginning \
  --property print.key=true \
  --property key.separator=' => '

# Read as a named consumer group
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic user-events \
  --group my-consumer-group \
  --from-beginning
```

Consumer Group Management

```
# List all consumer groups
docker exec -it kafka-kraft \
  kafka-consumer-groups --bootstrap-server localhost:9092 --list

# Describe a group (shows LAG — the important metric)
```

```
docker exec -it kafka-kraft \
  kafka-consumer-groups --bootstrap-server localhost:9092 \
  --describe --group my-consumer-group

# Output columns: GROUP, TOPIC, PARTITION, CURRENT-OFFSET, LOG-END-OFFSET, LAG
# LAG = LOG-END-OFFSET - CURRENT-OFFSET
# LAG > 0 means your consumer is behind — it has unprocessed messages!
```

 Consumer Lag: In production, LAG is the most important Kafka health metric. A growing lag means your consumers can't keep up with producers. Alert on this.

✓**PRACTICE TASK 2: Full Producer → Consumer Roundtrip**

Goal: Run two terminals and see real-time message flow with consumer groups.

Step 1 — Create a topic with 2 partitions:

```
docker exec -it kafka-kraft \
  kafka-topics --bootstrap-server localhost:9092 \
  --create --topic orders --partitions 2 --replication-factor 1
```

Step 2 — In terminal 1, start a producer with keys:

```
docker exec -it kafka-kraft \
  kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --property parse.key=true \
  --property key.separator=:
```

Step 3 — In terminal 2, start consumer group 'order-processors':

```
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --group order-processors \
  --from-beginning \
  --property print.key=true
```

Step 4 — In terminal 1, type messages:

```
> order-101:{"item": "laptop", "qty": 1}
> order-102:{"item": "mouse", "qty": 2}
> order-101:{"item": "keyboard", "qty": 1}
```

Step 5 — In terminal 3, check the consumer group lag:

```
docker exec -it kafka-kraft \
  kafka-consumer-groups --bootstrap-server localhost:9092 \
  --describe --group order-processors
```

Bonus: Open a second consumer in the same group. See how partitions are rebalanced between the two consumers.

PHASE 3

Java Producer & Consumer (Spring Boot)

Project Setup — Dependencies

Add to your pom.xml:

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>

<!-- Optional: for Avro/JSON serialization -->
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

application.yml:

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
      acks: all              # wait for all ISR to acknowledge
      retries: 3
      properties:
        enable.idempotence: true    # exactly-once producer semantics
    consumer:
      group-id: my-app-group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      auto-offset-reset: earliest  # start from beginning if no committed offset
      enable-auto-commit: false    # manual commit = safer
```

Java Producer

```
@Service
public class OrderProducer {

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;

    public OrderProducer(KafkaTemplate<String, String> kafkaTemplate,
                         ObjectMapper objectMapper) {
        this.kafkaTemplate = kafkaTemplate;
```

```
        this.objectMapper = objectMapper;
    }

    public void sendOrder(String orderId, OrderDto order) {
        try {
            String payload = objectMapper.writeValueAsString(order);
            // Key = orderId → same order always goes to same partition
            ProducerRecord<String, String> record =
                new ProducerRecord<>("orders", orderId, payload);

            kafkaTemplate.send(record)
                .whenComplete((result, ex) -> {
                    if (ex != null) {
                        log.error("Failed to send order {}", orderId, ex);
                    } else {
                        RecordMetadata meta = result.getRecordMetadata();
                        log.info("Order {} sent → topic={}, partition={},
offset={}",
                            orderId, meta.topic(), meta.partition(),
meta.offset());
                    }
                });
        } catch (JsonProcessingException e) {
            throw new RuntimeException("Serialization failed", e);
        }
    }
}
```

Java Consumer

```
@Component
public class OrderConsumer {

    @KafkaListener(
        topics = "orders",
        groupId = "order-processors",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void listen(
            @Payload String message,
            @Header(KafkaHeaders.RECEIVED_KEY) String key,
            @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
            @Header(KafkaHeaders.OFFSET) long offset,
            Acknowledgment acknowledgment) {

        try {
            log.info("Received key={} partition={} offset={}", key, partition,
offset);
            // ... process message ...
            acknowledgment.acknowledge();  // manual commit AFTER success
        } catch (Exception e) {
            log.error("Processing failed for key={}", key, e);
            // Don't acknowledge → message will be redelivered
        }
    }
```

```
}
```

 enable-auto-commit: false With manual acknowledgment, if your consumer crashes mid-processing, the offset is NOT committed. Kafka will redeliver the message to another consumer — no data loss. Auto-commit risks losing messages if the app crashes after commit but before processing.

KafkaListener Config Bean

```
@Configuration
public class KafkaConsumerConfig {

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, String>
           kafkaListenerContainerFactory(
               ConsumerFactory<String, String> consumerFactory) {

        var factory = new ConcurrentKafkaListenerContainerFactory<String,
String>();
        factory.setConsumerFactory(consumerFactory);
        factory.getContainerProperties()
               .setAckMode(ContainerProperties.AckMode.MANUAL_IMMEDIATE);
        factory.setConcurrency(3);  // 3 threads = up to 3 partitions in parallel
        return factory;
    }
}
```

✓**PRACTICE TASK 3: Spring Boot Producer + Consumer App**

Goal: Build a minimal Spring Boot app that produces and consumes JSON events.

What to build:

- OrderDto record: orderId, item, qty, timestamp
- OrderProducer: sends serialized JSON keyed by orderId
- OrderConsumer: deserializes and logs each received order with partition + offset
- REST endpoint POST /orders → triggers producer

Verification commands (run these while your app is running):

```
# Watch messages arriving in Kafka while your app produces
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning \
  --property print.key=true
```

```
# Check consumer group lag for your app's group
docker exec -it kafka-kraft \
  kafka-consumer-groups --bootstrap-server localhost:9092 \
  --describe --group order-processors
```

Challenge: Stop your Spring Boot app. Send 5 messages via CLI producer. Restart the app — it should replay the missed messages because enable-auto-commit=false and offsets were not committed.

PHASE 4

Delivery Guarantees & Error Handling

The Three Delivery Semantics

Semantic	Guarantee & Config
At-most-once	Message may be lost, never duplicated. acks=0, auto-commit=true. Fast but risky.
At-least-once	Message never lost, may be duplicated. acks=all, manual commit, retries. Default choice for most apps.
Exactly-once	No loss, no duplicates. Requires idempotent producer + transactions. Complex, use only when duplicates are truly unacceptable.

 Middle dev reality: Design your consumer to be IDEMPOTENT (processing the same message twice produces the same result). This lets you safely use at-least-once delivery and eliminates the complexity of exactly-once.

Producer Acknowledgment (acks)

```
# acks=0  → fire and forget (no confirmation, fastest, data loss possible)
# acks=1  → leader confirms write (leader crash before replication = data loss)
# acks=all → all ISR confirm write (safest, slightly slower)

# In application.yml:
spring.kafka.producer.acks: all
spring.kafka.producer.properties.enable.idempotence: true
# enable.idempotence=true forces acks=all automatically and adds
# sequence numbers to prevent duplicate writes on retry
```

Dead Letter Topic (DLT) Pattern

When a consumer fails to process a message after all retries, the message should go to a Dead Letter Topic for later inspection — not be silently dropped.

```
@Configuration
public class KafkaErrorConfig {

    @Bean
    public DefaultErrorHandler errorHandler(
            KafkaTemplate<String, String> template) {

        // Publish failed messages to <topic-name>.DLT
```

```
        var recoverer = new DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(
                record.topic() + ".DLT",
                record.partition()  // keep same partition for ordering
            ));

        // Retry 3 times with 1s backoff before sending to DLT
        var backoff = new FixedBackOff(1000L, 3);
        var handler = new DefaultErrorHandler(recoverer, backoff);

        // Don't retry on deserialization errors (permanent failures)
        handler.addNotRetryableExceptions(DeserializationException.class);

        return handler;
    }
}
```

```
# Monitor your DLT in CLI:
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders.DLT \
  --from-beginning \
  --property print.headers=true

# Headers will show: kafka_dlt-exception-message, kafka_dlt-original-offset, etc.
```

Offset Reset Strategies

```
# auto-offset-reset applies when a consumer group has NO committed offset yet

# earliest → read from the very beginning of the topic
spring.kafka.consumer.auto-offset-reset: earliest

# latest → skip all existing messages, read only new ones
spring.kafka.consumer.auto-offset-reset: latest

# Reset a consumer group manually (while consumers are STOPPED):
docker exec -it kafka-kraft \
  kafka-consumer-groups --bootstrap-server localhost:9092 \
  --group order-processors \
  --topic orders \
  --reset-offsets --to-earliest --execute

# This lets you replay all events — powerful for debugging or data recovery
```

✅**PRACTICE TASK 4: Implement DLT + Retry Logic**
Goal: Force a consumer failure and observe the DLT in action.

Setup:

- Add DeadLetterPublishingRecoverer config as shown above
- Create orders.DLT topic: partitions=2, replication-factor=1
- Add a DLT consumer that logs DLT messages with full headers

Simulate failure:

```
// In your OrderConsumer, throw an exception for a specific order:
if (order.getItem().equals("FAIL")) {
    throw new RuntimeException("Simulated processing failure");
}
```

Send the poisonous message:

```
docker exec -it kafka-kraft \
  kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --property parse.key=true --property key.separator=:

> order-999:{"orderId":"999","item":"FAIL","qty":1}
```

Observe: Watch your app retry 3 times with 1s backoff, then publish to orders.DLT. Consume from orders.DLT and inspect the exception headers.

PHASE 5

Kafka Streams Basics

What Are Kafka Streams?

Kafka Streams is a client library for building real-time stream processing applications. It reads from topics, transforms/aggregates/joins the data, and writes back to topics — all within a regular Java application. No separate cluster needed.

 vs Spring Kafka: Spring Kafka (@KafkaListener) = stateless record-by-record processing. Kafka Streams = stateful, windowed, joinable stream processing with built-in fault tolerance.

Kafka Streams Topology Concepts

Concept	Description
Topology	The processing graph: sources → processors → sinks.
KStream	Unbounded stream of records. Each record is independent (like a log).
KTable	Changelog stream — latest value per key (like a database table).
GlobalKTable	Like KTable but replicated to all instances (for lookups/joins).
Processor	Node in the topology. Can be filter, map, flatMap, groupBy, aggregate, join.
State Store	Local RocksDB storage for aggregations. Backed up to a changelog topic.
Windowing	Group records by time: Tumbling (fixed), Hopping (overlapping), Session (activity-based).

Streams Setup (Spring Boot)

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-streams</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

```
spring:
  kafka:
```

```
    streams:
      application-id: order-stream-app   # unique per app, used as consumer group
ID
      bootstrap-servers: localhost:9092
      properties:
        default.key.serde:
org.apache.kafka.common.serialization.Serdes$StringSerde
        default.value.serde:
org.apache.kafka.common.serialization.Serdes$StringSerde
```

Example: Count Orders per User (Aggregation)

```
@Configuration
@EnableKafkaStreams
public class OrderStreamTopology {

    @Bean
    public KStream<String, String> orderCountStream(StreamsBuilder builder) {

        KStream<String, String> orders = builder.stream("orders");

        // Count how many orders each user (key) placed
        KTable<String, Long> orderCounts = orders
            .groupByKey()                          // group by existing key
            .count(Materialized.as("order-count-store"));  // stateful count

        // Write results to output topic
        orderCounts.toStream().to("order-counts",
            Produced.with(Serdes.String(), Serdes.Long()));

        return orders;
    }
}
```

Example: Filter + Transform Stream

```
@Bean
public KStream<String, String> highValueOrders(StreamsBuilder builder) {
    ObjectMapper mapper = new ObjectMapper();

    return builder.stream("orders")
        .filter((key, value) -> {
            try {
                JsonNode node = mapper.readTree(value);
                return node.get("qty").asInt() > 5;  // only large orders
            } catch (Exception e) { return false; }
        })
        .mapValues(value -> "[HIGH-VALUE] " + value)
        .peek((k, v) -> log.info("High value order: {}", k))
        .through("high-value-orders");  // write to new topic AND return stream
}
```

✓PRACTICE TASK 5: Build an Order Count Stream

Goal: Create a Streams topology that counts orders per orderId key.

Steps:

1. Create topic 'order-counts' with 2 partitions
2. Build the KTable count topology as shown above
3. Start your app, produce multiple orders with the same key
4. Read from order-counts topic and see the running count

Verification:

```
# Create output topic first
docker exec -it kafka-kraft \
  kafka-topics --bootstrap-server localhost:9092 \
  --create --topic order-counts --partitions 2 --replication-factor 1

# Watch counts update in real time
docker exec -it kafka-kraft \
  kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic order-counts \
  --from-beginning \
  --property print.key=true \
  --value-deserializer org.apache.kafka.common.serialization.LongDeserializer
```

Bonus challenge: Add a tumbling window of 60 seconds. Show the count of orders per user PER MINUTE.

PHASE 6

Conductor UI, Performance & Middle-Level Topics

Using Conductor UI (Your Docker Stack with Postgres)

Your second Docker image includes Conduktor — a GUI for managing and monitoring Kafka. Use it alongside CLI commands to build intuition.

```
# Start the full stack (Kafka + Conduktor + Postgres)
# (adjust to your actual docker-compose file name)
docker-compose -f docker-compose-conduktor.yml up -d

# Conduktor UI is typically at:
# http://localhost:8080

# Default credentials (check your compose file):
# admin@conduktor.io / admin
```

Key things to explore in Conduktor UI:

- Topic Browser → see partitions, offsets, messages with JSON pretty-print
- Consumer Groups → visual lag chart per partition
- Produce messages → test your topics without CLI
- Schema Registry → if you add Avro/JSON Schema later

Partitioning Strategy — Important for Middle Level

Partition assignment determines throughput, ordering, and parallelism.

Strategy	When to use
Default (key hash)	Always use when ordering per key is needed (e.g. events per userId).
Round-robin (null key)	Maximum spread, no ordering guarantee. Good for independent events.
Custom partitioner	Implement Partitioner interface for business-logic routing (e.g. VIP users → partition 0).

```
// Custom Partitioner example
public class VipPartitioner implements Partitioner {
    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                         Object value, byte[] valueBytes, Cluster cluster) {
        int numPartitions = cluster.partitionCountForTopic(topic);
        // VIP orders always go to partition 0
```

```
        if (key != null && key.toString().startsWith("VIP-")) return 0;
        // Others distributed evenly across remaining partitions
        return (Math.abs(key.toString().hashCode()) % (numPartitions - 1)) + 1;
    }
}

// Register in producer config:
spring.kafka.producer.properties.partitioner.class=com.vbforge.VipPartitioner
```

Key Performance Tuning — Know the Knobs

Config	What it does
batch.size (default 16KB)	Producer batches records. Larger = fewer requests = higher throughput.
linger.ms (default 0)	Wait up to N ms to fill a batch before sending. linger.ms=5 improves throughput.
compression.type	snappy or lz4 recommended — reduces network + disk I/O significantly.
max.poll.records (default 500)	Max records per poll() call. Lower = faster per-batch processing; higher = more throughput.
fetch.min.bytes	Consumer waits until N bytes available before returning. Reduces idle polls.
session.timeout.ms	If broker doesn't hear from consumer within this time → rebalance triggered.

Rebalance — What Every Middle Dev Must Know

A rebalance is triggered when: a consumer joins or leaves a group, a consumer fails to heartbeat, topic partition count changes.

Problem: During rebalance, ALL consumers stop processing. This causes latency spikes.

Modern solution — Cooperative Sticky Rebalancing:

```
# application.yml
spring:
  kafka:
    consumer:
      properties:
        partition.assignment.strategy: \
          org.apache.kafka.clients.consumer.CooperativeStickyAssignor

# Result: Only partitions that NEED to move are reassigned.
# Other partitions keep processing during rebalance.
# This is the production-grade setting.
```

Testing Kafka in Spring Boot

```
// Use @EmbeddedKafka for integration tests — no Docker needed
@SpringBootTest
@EmbeddedKafka(
    partitions = 2,
    topics = {"orders", "orders.DLT"},
    bootstrapServersProperty = "spring.kafka.bootstrap-servers"
)
class OrderConsumerIntegrationTest {

    @Autowired private KafkaTemplate<String, String> kafkaTemplate;
    @Autowired private OrderConsumer consumer;

    @Test
    void givenValidOrder_whenProduced_thenConsumed() throws Exception {
        String orderId = "test-001";
        String payload = "{\"item\":\"laptop\",\"qty\":1}";

        kafkaTemplate.send("orders", orderId, payload).get(5, TimeUnit.SECONDS);

        // Use CountDownLatch or Awaitility to wait for async consumption
        await().atMost(10, SECONDS)
               .untilAsserted(() ->
                   verify(consumer, times(1)).listen(eq(payload), any(), any(),
any(), any())
               );
    }
}
```

✓**PRACTICE TASK 6: Full End-to-End Portfolio Feature**

Goal: Build a complete mini system that demonstrates middle-level Kafka skills.

Build this:

5. **Order Service** — REST API → produces to 'orders' topic with JSON payload
6. **Inventory Stream** — Kafka Streams topology: filter qty > 0, count orders per item, output to 'inventory-summary'
7. **Notification Service** — @KafkaListener on 'orders', manual ack, DLT on 3 failures, logs each order with partition+offset
8. **Dead Letter Monitor** — separate listener on 'orders.DLT', stores failed orders to a list (in-memory or DB)

Use Conduktor UI to:

- Monitor consumer group lag in real time
- Inspect messages on all 4 topics visually
- Trigger a manual rebalance by stopping/starting consumer instances

Add @EmbeddedKafka integration test for the Notification Service that verifies DLT routing on failure.

Commit to GitHub. This is a portfolio-worthy project that demonstrates real-world Kafka patterns.

Quick Reference Cheat Sheet

Essential CLI Commands

```
# ── TOPICS ──────────────────────────────────────────────────────
kafka-topics --bootstrap-server localhost:9092 --list
kafka-topics --bootstrap-server localhost:9092 --create --topic NAME --partitions
N --replication-factor 1
kafka-topics --bootstrap-server localhost:9092 --describe --topic NAME
kafka-topics --bootstrap-server localhost:9092 --delete --topic NAME

# ── PRODUCER ────────────────────────────────────────────────────
kafka-console-producer --bootstrap-server localhost:9092 --topic NAME
kafka-console-producer ... --property parse.key=true --property key.separator=:

# ── CONSUMER ────────────────────────────────────────────────────
kafka-console-consumer --bootstrap-server localhost:9092 --topic NAME --from-
beginning
kafka-console-consumer ... --group GROUP_NAME
kafka-console-consumer ... --property print.key=true --property key.separator=' => 
'

# ── CONSUMER GROUPS ─────────────────────────────────────────────
kafka-consumer-groups --bootstrap-server localhost:9092 --list
kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group GROUP
kafka-consumer-groups --bootstrap-server localhost:9092 --group GROUP --topic
TOPIC
  --reset-offsets --to-earliest --execute

# ── WRAP IN docker exec ─────────────────────────────────────────
docker exec -it kafka-kraft <command above>
```

Config Quick Reference

Property	Recommended Value / Notes
acks	all — safest, requires all ISR to confirm
enable.idempotence	true — prevents duplicate writes on retry
retries	3 — retry transient failures
auto-offset-reset	earliest (dev) / latest (prod new groups)
enable-auto-commit	false — use manual Acknowledgment
max.poll.records	100-500 — tune per message processing time
compression.type	snappy or lz4 in production
partition.assignment.strategy	CooperativeStickyAssignor — production grade
linger.ms	5-20ms — batch producer messages for throughput

Mental Models to Internalize

- **Partition = unit of parallelism.** More partitions = more consumers. Can't go below existing partition count.
- **Key = routing.** Same key → same partition → guaranteed ordering for that key.
- **Consumer group = logical subscriber.** Multiple apps can consume the same topic independently with different group IDs.
- **Offset = progress tracker.** Commit it only after successful processing. Never before.
- **Retention = replay window.** Events stay on disk until retention expires. You can replay from any point.
- **Lag = health indicator.** Growing lag = consumer is slower than producer. Alert and scale.
- **KRaft = no ZooKeeper.** Raft consensus inside Kafka brokers. Simpler ops, higher partition limits.